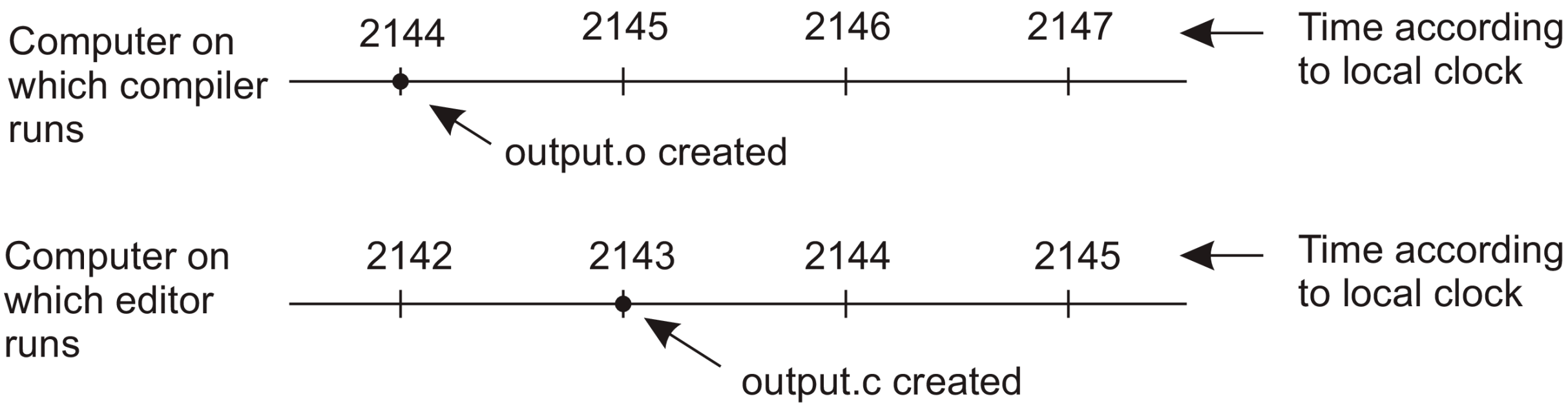
# Synchronization

- Synchronization needed to cooperate processes
  - Access resources
  - Agree on ordering of events
- More challenging than synchronization in a distributed system
- Outline
  - Synchronization based on actual time
  - Synchronization based on logical time

# Clock Synchronization Needed




- When each machine has its own clock, an event that occurred after another event may nevertheless be assigned an earlier time.

# Physical Clocks (1)

- Clock -> timer: precisely machined quartz crystal
- A counter register and holding register
  - Each oscillation decrements the counter by one
  - Interrupt generated when counter register reaches 0
  - Reloaded from the holding register
  - Interrupt: clock tick
- Clock skew: caused by ticks generated at different rates
  - Because, in practice, crystals will run at slightly different rates

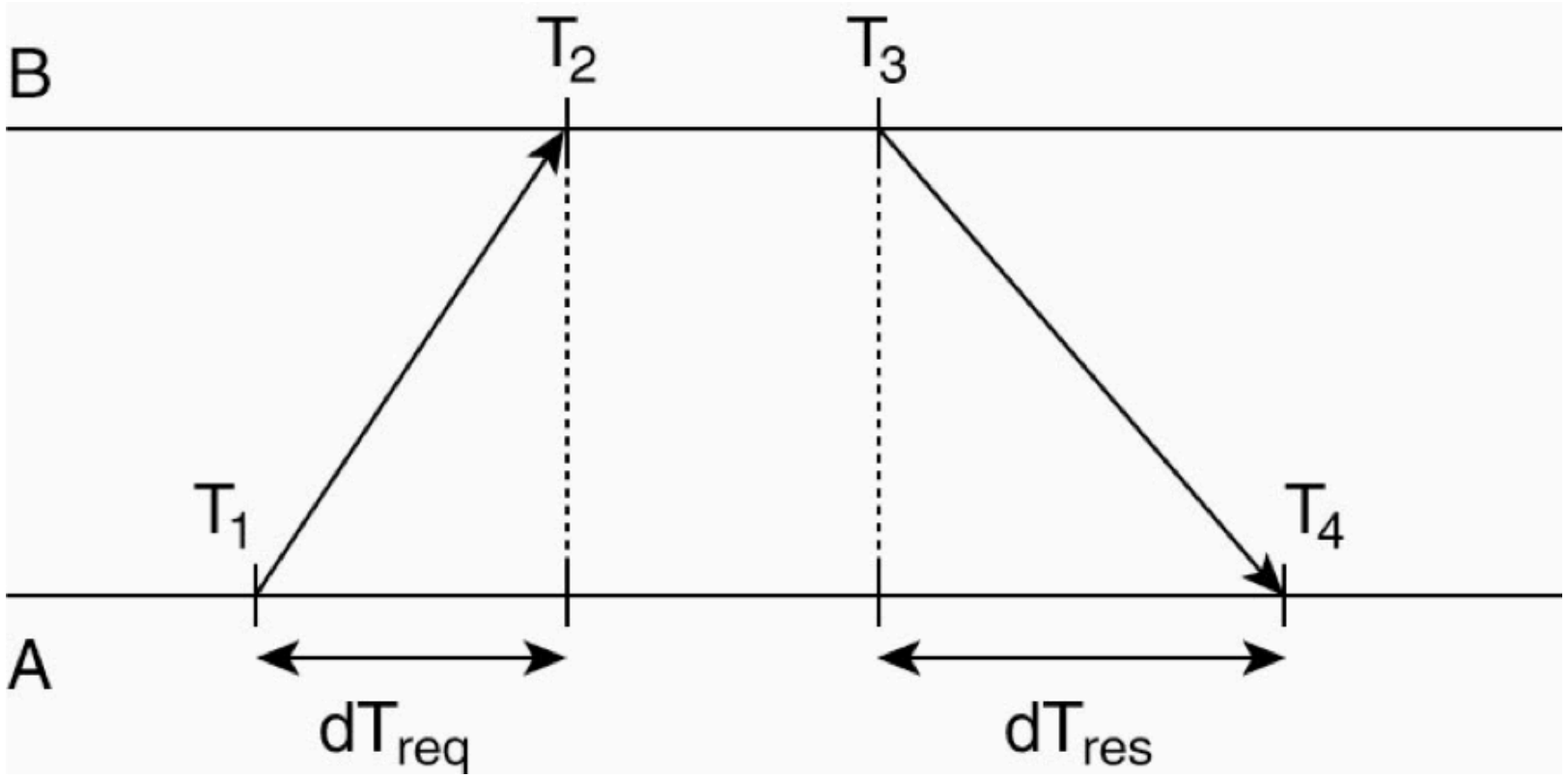
# Physical Clocks (2)

- International Atomic Time (TAI)
  - Defined based on transitions of the cesium 133 atom
  - 1 second = 9,192,631,770 transitions
- Universal Coordinated Time (UTC)
  - Solar day getting longer than TAI
  - Introduced corrections to TAI (leap seconds)
  - WWV station broadcasts UTC every second (with ±1 msec)
  - Satellites provide UTC services too (with ±0.5 msec)

# Clock Synchronization: Network Time Protocol (1)

- Poll and synchronize with a time server
- Time server: equipped with a WWV/satellite receiver
- Messages have delays and can have outdated reported time
- Need to estimate delays

# Clock Synchronization: Network Time Protocol (2)




- Offset estimated as: $\theta = \frac{(T_2 - T_1) + (T_3 - T_4)}{2}$

# Clock Synchronization: Network Time Protocol (3)

- Iff A's clock faster ($\theta<0$), can't make time backwards usually
  - Solution: change time gradually
  - If the timer is set to generate 100 interrupts per second.
  - Normally, each interrupt would add 10 msec to the time.
  - When slowing down, the interrupt routine adds only 9 msec each time until the correction has been made.

# Logical Clocks

- Synchronization discussed so far based on actual time
  - Can agree on a real time
- Not necessary to agree on a time some times
  - if two processes do not interact, it is not necessary that their clocks be synchronized
  - the lack of synchronization would not be observable and thus could not cause problems
- More important to agree the order of events
  - Logical clocks
  - Leslie Lamport 1978 paper “Time, clocks, and the ordering of events in a distributed system”

# Lamport’s Logical Clocks (1)

- The "happens-before" relation → can be observed directly in two situations:
- If `a` and `b` are events in the same process, and `a` occurs before `b`, then `a` → `b` is true.
- If `a` is the event of a message being sent by one process, and `b` is the event of the message being received by another process, then `a` → `b`

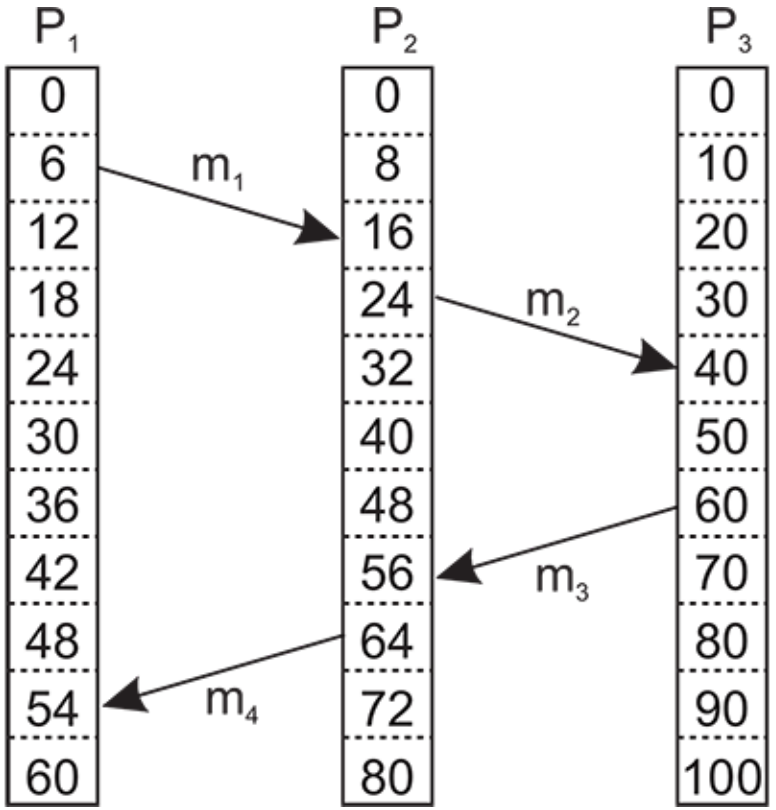
# Lamport's Logical Clocks (2)

- `Happens-before` is transitive
  - If a -> b and b -> c, then a -> c
- `Concurrent`
  - If two events, x and y, happen in different processes and do not exchange messages, then x -> y is not true, neither y -> x

# Lamport’s Logical Clocks (3)

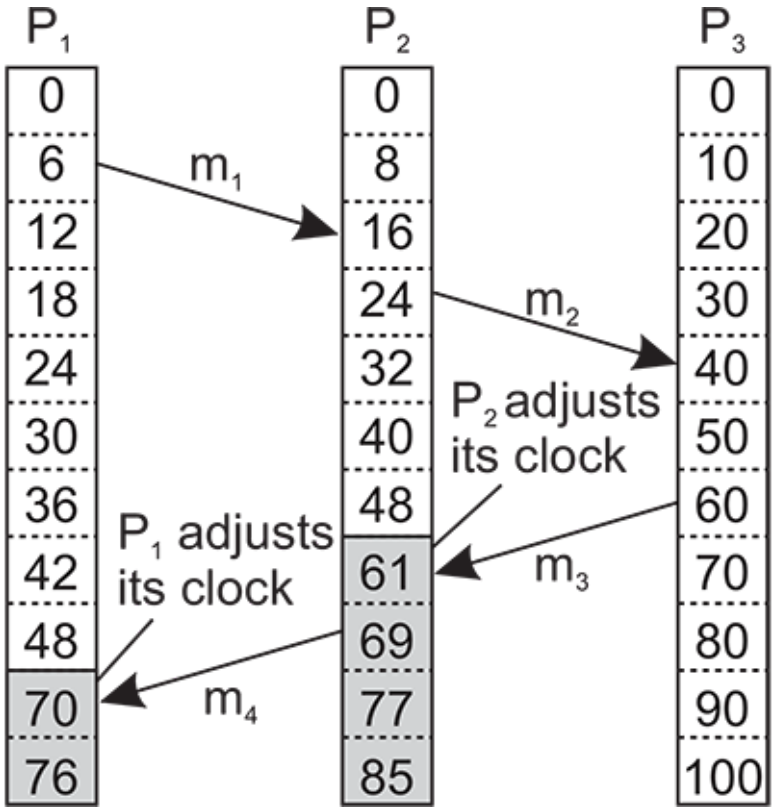
- Synchronize logical clocks: assign every event, `a`, a time value `C(a)` so that
  - If a -> b, C(a) < C(b)
  - Always go forward not backward (can only add a positive value to correct time, not subtracting one)

# Lamport's Logical Clocks (4)



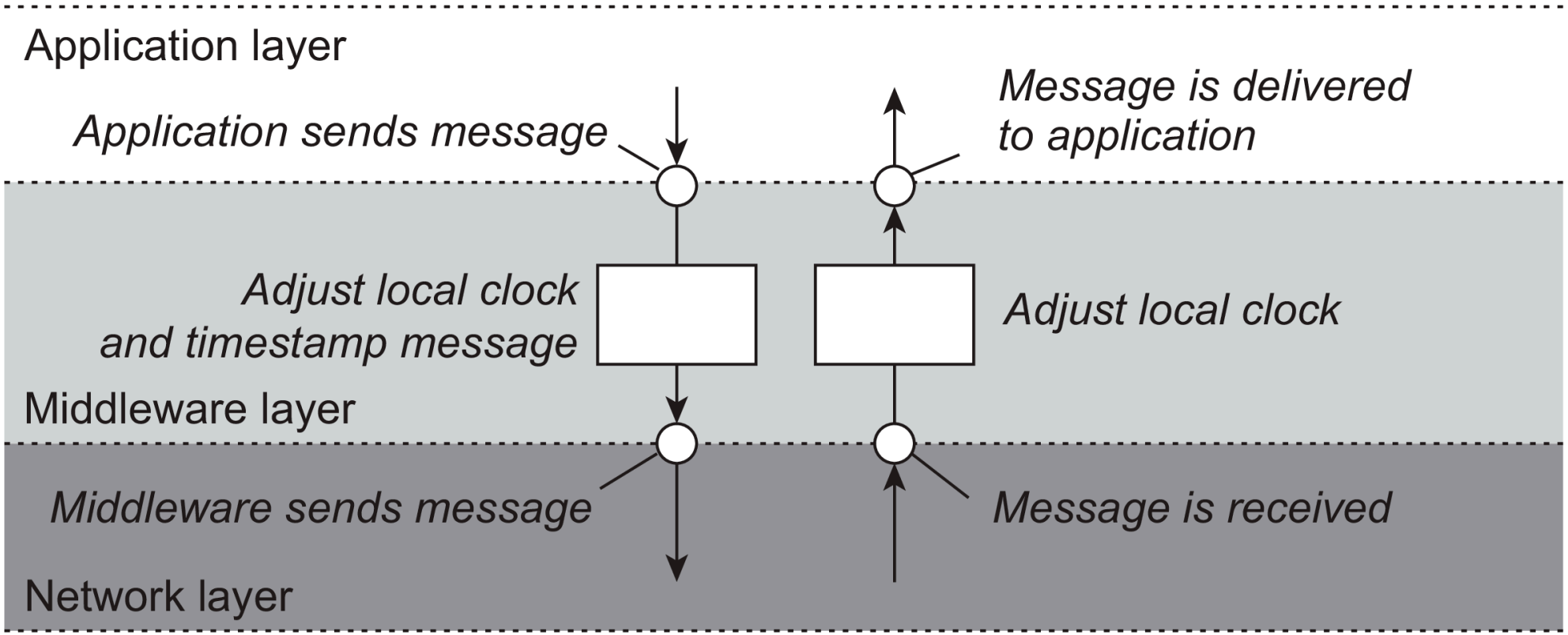

- Three processes, each with its own clock.
- The clocks run at different rates.
- P1, P2, and P3 tick 6, 8, and 10 times respectively

# Lamport's Logical Clocks (5)




- Lamport's algorithm corrects the clocks.
- Message carries sending time according to sender's clock
- Receiver advances clock if clock value prior to message sent

# Lamport's Logical Clocks (6)




The positioning of Lamport's logical clocks in distributed systems.

# Lamport’s Logical Clocks (7)

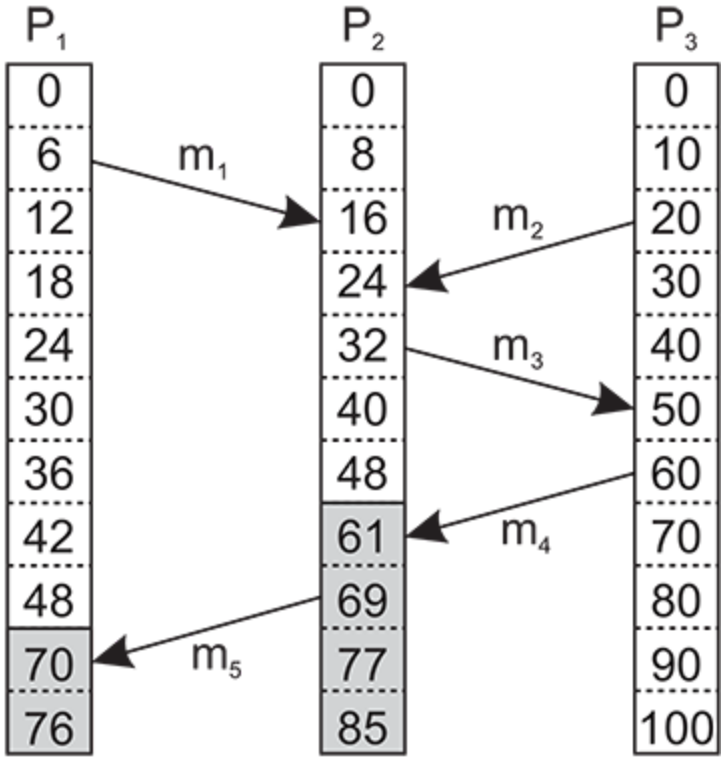
Implementation (each process `Pi` maintains a local counter `Ci`)

1. Before executing an event (i.e., sending a message over the network, delivering a message to an application, or some other internal event), `Pi` executes `Ci = Ci+1`.
2. When process `Pi` sends a message `m` to `Pj`, it sets m's timestamp `ts(m)` equal to `Ci` after having executed the previous step.
3. Upon the receipt of a message m, process `Pj` adjusts its own local counter as `Cj = max{Cj,ts(m)}`, after which it then executes the first step and delivers the message to the application.

# Vector Clocks (1)

- Lamport's logical clocks lead to, if event a happened before `b`, then `C(a) < C(b)`
- But nothing can be said about the relationship between two events `a` and `b` by comparing time values
  - If `C(a) < C(b)`, it doesn't necessarily imply `a` indeed before `b`
  - Because Lamport clocks do not capture causality
- Vector clocks introduced to capture causality

# Vector Clocks (1)




- We know relationship between $m_1$ and $m_3$, but we do not know the relationship between $m_1$ and $m_2$, even we know C($m_1$) < C($m_2$).

# Vector Clocks (2)

Vector clocks are constructed by letting each process `P_i` maintain a vector `VC_i` with the following two properties:

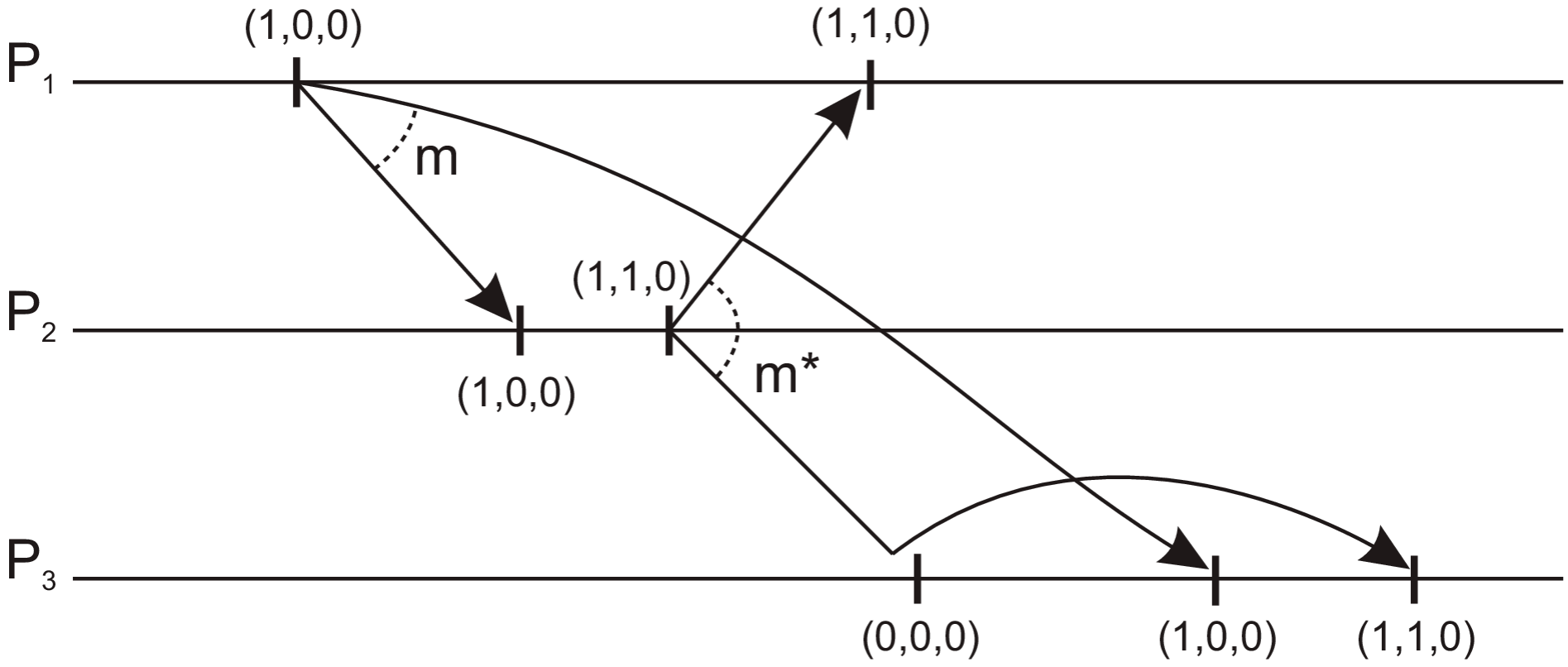
- `VC_i[i]` is the number of events that have occurred so far at `P_i`. In other words, `VC_i[i]` is the local logical clock at process `P_i`
- If `VC_i[j] = k` then `P_i` knows that `k` events have occurred at `P_j`. It is thus `P_i`'s knowledge of the local time at `P_j`.

# Vector Clocks (3)

Steps carried out to accomplish property 2 of previous slide:

- Before executing an event `P_i` executes `VC_i [i]` ← `VC_i[i]` + 1.
- When process `P_i` sends a message `m` to `P_j`, it sets m's (vector) timestamp `ts(m)` equal to `VC_i` after having executed the previous step.
- Upon the receipt of a message `m`, process `P_j` adjusts its own vector by setting `VC_j [k]` ← `max{VC_j [k], ts(m)[k]}` for each `k`, after which it executes the first step and delivers the message to the application.

# Vector Clocks (4)



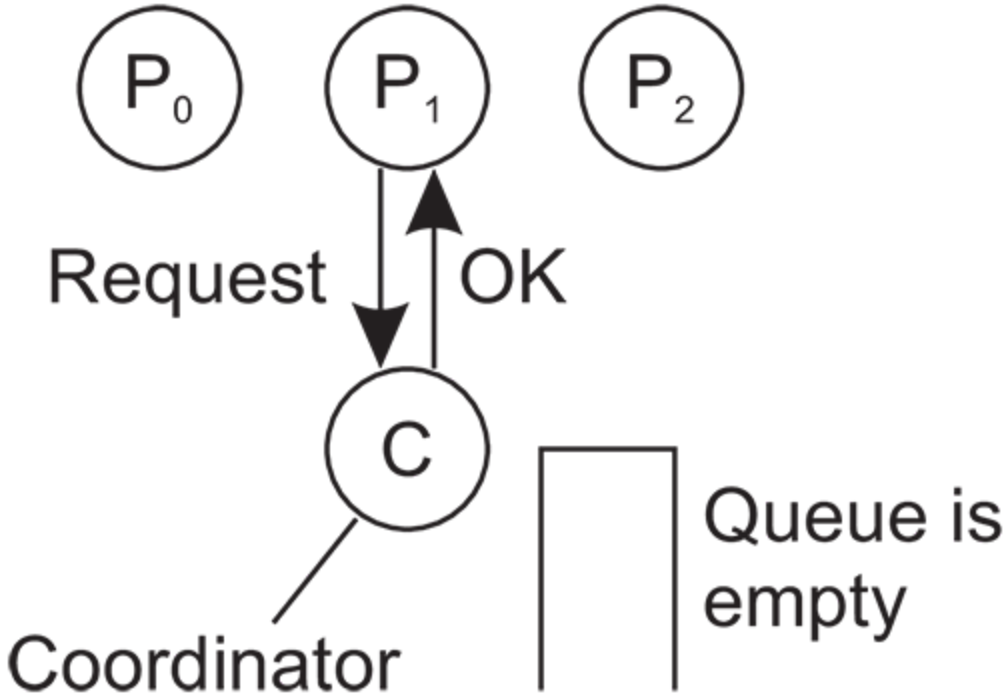
# Mutual Exclusion (1)

- Mutual exclusion needed to avoid inconsistent state when accessing shared resources
  - Critical region or critical section
- Requirements
  - No two processes simultaneously inside critical regions
  - No deadlocks should occur
  - No process running outside its critical region may block other processes
  - No process should have to wait forever to enter its critical region (starvation)

# Mutual Exclusion (2)

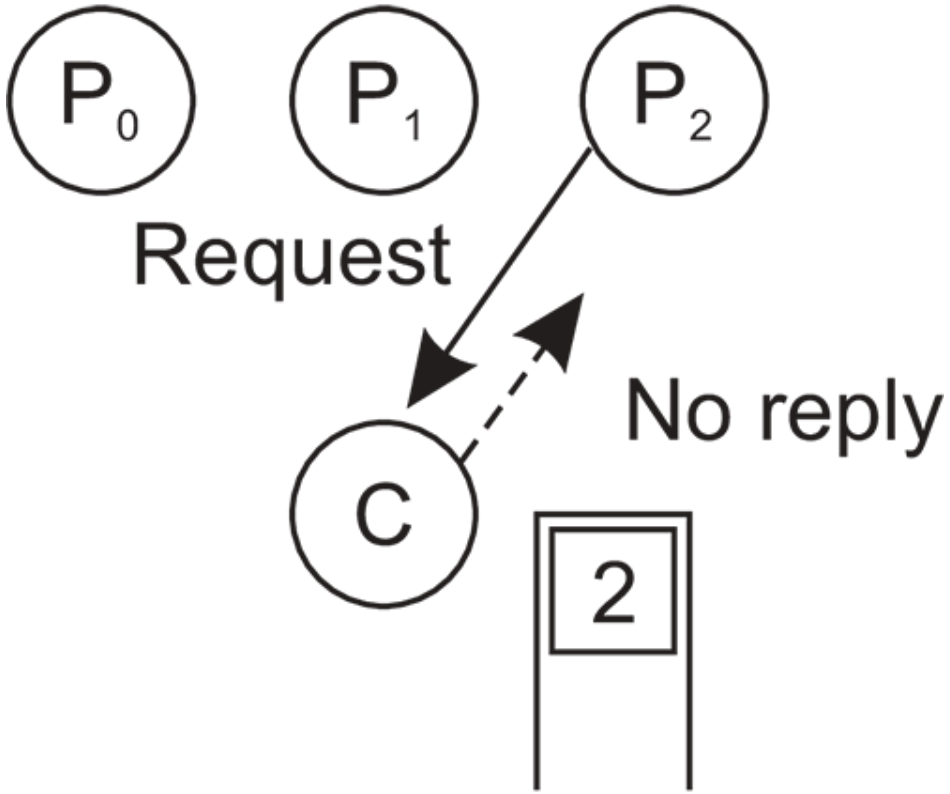
- Permission-based approach
  - Centralized algorithm
  - Decentralized algorithm (skipped)
  - Distributed algorithm
- Token-based approach
  - Token-ring algorithm

# A Centralized Algorithm (1)



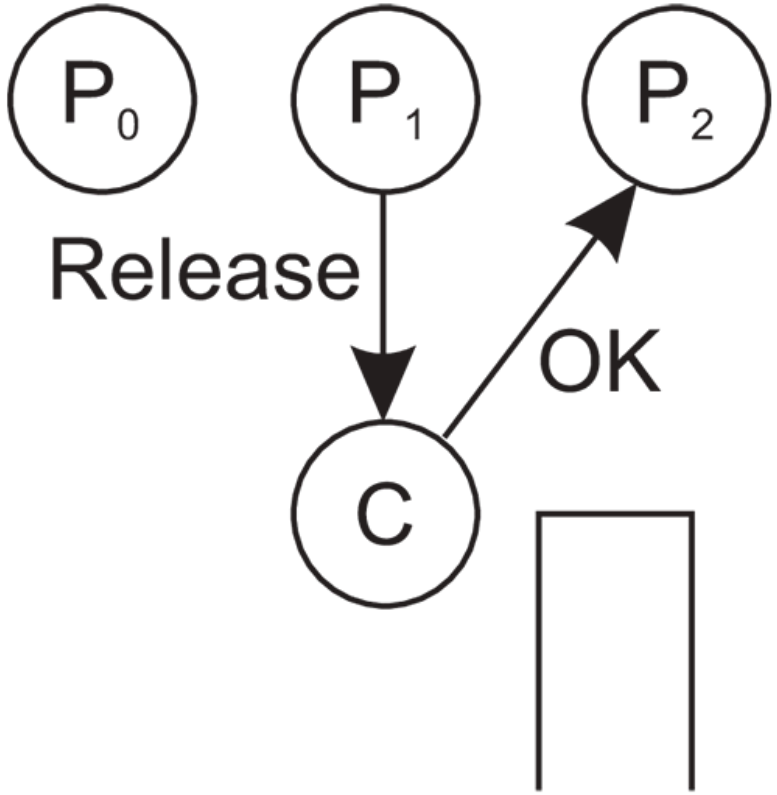

Process 1 asks the coordinator for permission to access a shared resource. Permission is granted.

# A Centralized Algorithm (2)




Process 2 then asks permission to access the same resource. The coordinator does not reply.

# A Centralized Algorithm (3)




When process 1 releases the resource, it tells the coordinator, which then replies to 2.

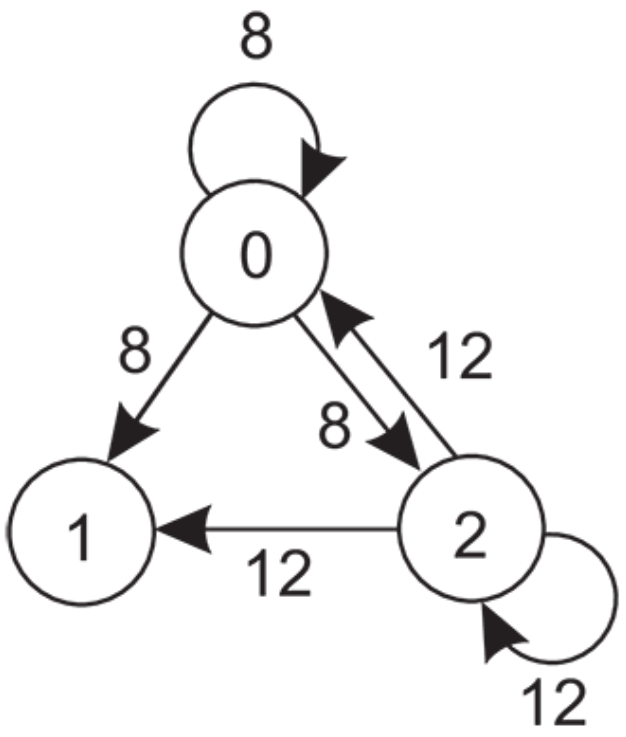
# A Distributed Algorithm (1)

- Desire fully distributed and deterministic algorithm
- Require ordering of events (logical clock)
- Algorithm
  - Requesting process broadcasts a message with a timestamp to all processes
  - Wait till OK msgs from all other processes, access resource, then sends OK msg to all processes in its queue
  - Processes receiving a request take the following action

# A Distributed Algorithm (2)

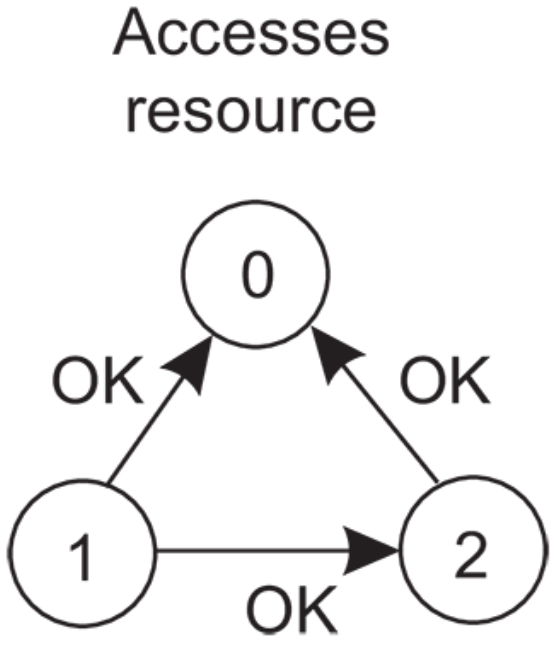
- Three different cases:
  - If the receiver is not accessing the resource and does not want to access it, it sends back an OK message to the sender.
  - If the receiver already has access to the resource, it simply does not reply. Instead, it queues the request.
  - If the receiver wants to access the resource as well but has not yet done so, it compares the timestamp of the incoming message with the one contained in the message that it has sent everyone. The lowest one wins.

# A Distributed Algorithm (3)



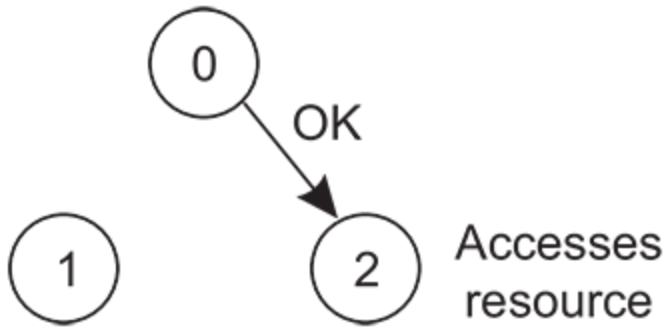

Two processes want to access a shared resource at the same moment.

# A Distributed Algorithm (4)




Process 0 has the lowest timestamp. so it wins.

# A Distributed Algorithm (5)




When process 0 is done, it sends an OK also, so 2 can now go ahead.

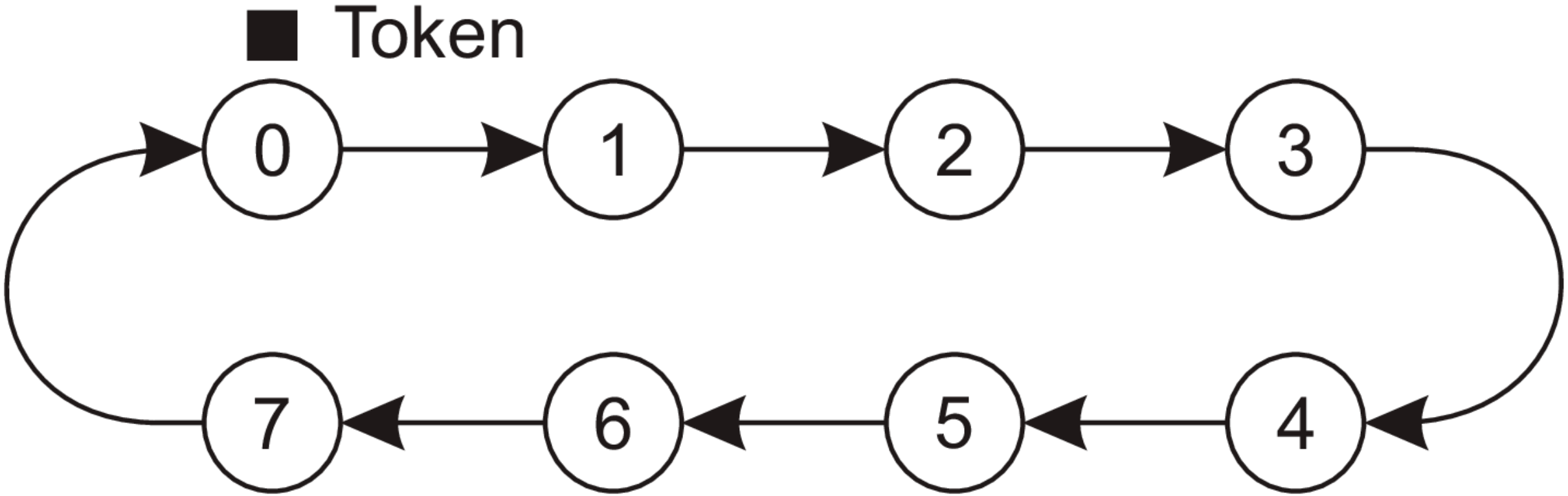
# A Distributed Algorithm (6)

Issues?

- Single point failure -> n point failure…
- Multicast communication is required…

# A Token Ring Algorithm (1)

- Organize processes as a logical ring, with each process assigned a position in the ring
- Can be organized based on the order of network addr., etc.
- Each process knows its successor



# A Token Ring Algorithm (2)

- A token circulates around the ring (a point-to-point message from process k to k+1 (mod ring size)
- Only the process who owns the token can access the resource
- If the process with the token not interested in accessing the resource, it passes the token along

# A Token Ring Algorithm (3)

- If the token is ever lost, it must be regenerated.
  - detecting that it is lost is difficult
- If a process crashes, it must be recovered.
  - If we require a process receiving the token to acknowledge receipt,
  - a dead process will be detected when its neighbor tries to give it the token and fails.

# Fault Tolerance — Road Map

Distributed systems fail **partially**: one node crashes, one packet is dropped, one reply is late — while everything else keeps running. This section builds fault tolerance in **four layers**, then ends with a **contrast branch**:

**Part 1 — Dependability vocabulary and failure models**
→ Availability / Reliability / Safety / Maintainability, fault vs. error vs. failure, redundancy

**Part 2 — Reliable communication under crash / omission faults**
→ Lost requests, lost replies, server crashes mid-RPC, retry, timeout, deduplication

**Part 3 — Atomic commit across participants**
→ All commit or all abort; 2PC can block; 3PC helps only under stronger timing assumptions

**Part 4 — Crash-fault consensus for replicated state**
→ Paxos / Multi-Paxos choose one history for a replica group and keep a control plane available

**Contrast branch — Byzantine faults**
→ If nodes may lie or equivocate, we need a different protocol family and higher replication cost

# Dependability: Four Properties

Being fault tolerant is strongly related to **dependable systems**. Dependability has four dimensions:

| Property | Definition | Key Metric |
|---|---|---|
| **Availability** | System ready to be used at a given *instant* | Probability of working *right now* (e.g., 99.9%) |
| **Reliability** | System runs continuously without failure over a *time interval* | Mean time between failures (MTBF) |
| **Safety** | Nothing catastrophic when system temporarily fails | No dangerous side-effects on failure |
| **Maintainability** | How easily a failed system can be repaired | Mean time to repair (MTTR); auto-recovery |

> Availability = instant; Reliability = interval. A system can have high availability but low reliability if it fails often but recovers fast.

# Fault, Error, and Failure

- **Fault**: the cause — buggy code, hardware defect, misconfiguration, cosmic ray
- **Error**: the bad internal state created by that fault
- **Failure**: the externally visible violation of the system specification

```
Fault  ->  Error  ->  Failure
cause      state      observable symptom
```

- Fault tolerance tries to stop this chain before users observe a failure.

# Failure Models

| Type | What happens | Severity |
|---|---|---|
| **Crash** | Server halts; works correctly until it does | Easiest |
| **Omission** | Fails to receive or send messages (network drop) | ↑ |
| **Timing** | Response arrives outside the expected time window | ↑ |
| **Response** | Returns a wrong value or takes a wrong state transition | ↑ |
| **Byzantine** (Arbitrary) | May send anything, including different lies to different peers | Hardest |

> This course covers **crash / omission** (Reliable Comm + 2PC/Paxos) and **Byzantine** (Byzantine Agreement). Timing and response failures are out of scope.

# From Failure Model to Protocol Family

| If the system must preserve... | under faults such as... | typical mechanism |
|---|---|---|
| Correct request/response behavior | crash, omission, delay | TCP, RPC timeout/retry, deduplication |
| All-or-none update across nodes | crash, omission | 2PC, 3PC |
| One decision across replicas | crash-stop + asynchronous messaging | Paxos / Multi-Paxos |
| Agreement despite arbitrary behavior | Byzantine faults | Byzantine Agreement / BFT |

# Reliable Communication Under Crash/Omission Faults

- First main question: how do we make **request/response** communication correct enough to build larger protocols on top?
- Messages may be lost, duplicated, delayed, or arrive after a timeout.
- A server may execute an operation even when the client never sees the reply.

> So the real issue is not just packet delivery; it is the **semantic meaning of retry and timeout**.

# Reliable Communication

- Distributed systems must consider communication failures in addition to node failures.
- For communication channels, the most common behaviors are:
  - **omission failures** — messages are lost in one or both directions
  - **timing failures** — messages arrive too late to be useful
- For endpoints, **crash failures** are still part of the RPC story.
- In practice, crash + omission failures dominate the basic RPC discussion.

# Point-to-Point Communication

- Reliable point-to-point communication is usually built on a transport protocol such as TCP.
- TCP hides many omission failures with acknowledgments and retransmissions.
- But TCP does **not** solve application-level uncertainty:
  - the peer may crash
  - the request may have executed before the connection broke
  - the client may see only a timeout

# RPC Failure Cases

- Five classic failure cases in RPC systems:
  - The client cannot locate the server.
  - The request message is lost.
  - The server crashes after receiving the request.
  - The reply message is lost.
  - The client crashes after sending the request.

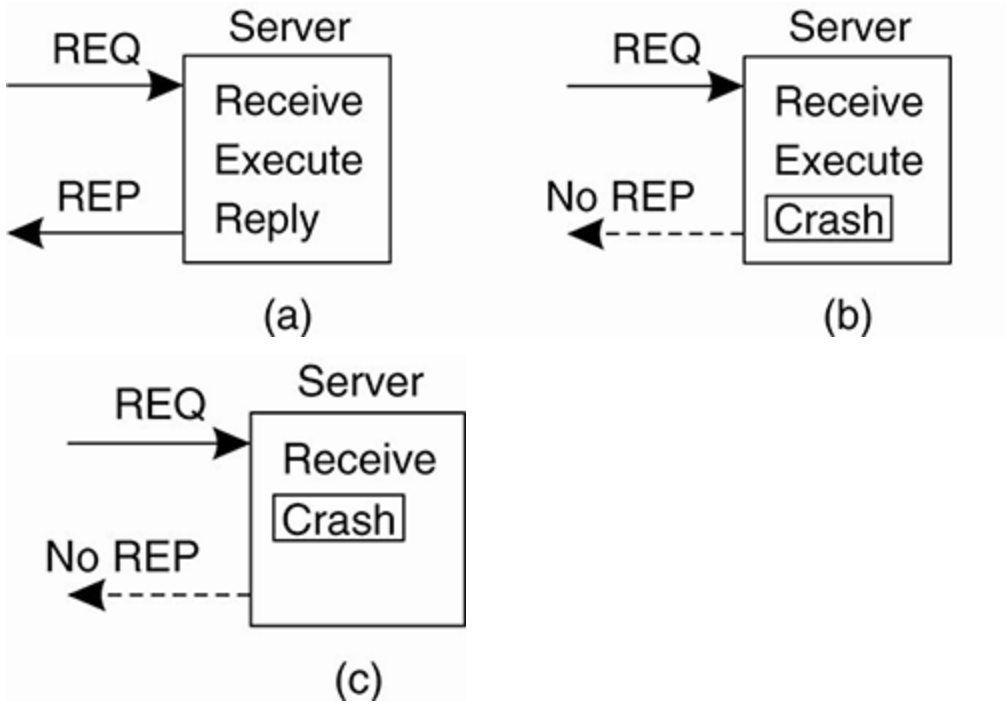
## [1] Client Cannot Locate Server

- Can happen if all servers are down or if client/server stubs do not match.
- A common response is to raise an exception.
- This reduces transparency, but it is better than pretending the call succeeded.

## [2] Lost Request Messages

- Associate each request with a timer.
- If the timer expires before an acknowledgment or reply arrives, retransmit.
- After too many retries, report failure to the caller.

# [3] Server Crashes




- Crash **before** execution: retry is usually safe.
- Crash **after** execution but before reply: retry may duplicate the side effect.
- The client cannot distinguish these cases from timeout alone.

# RPC Semantics: What Can We Promise?

| Policy | Meaning | Trade-off |
|---|---|---|
| **At-least-once** | Keep retrying until a reply arrives | Operation may execute more than once |
| **At-most-once** | Avoid duplicate execution by refusing ambiguous retries | A completed operation may look like failure |
| **Deduplicated execution** | Use request IDs and durable dedup state | Stronger guarantee, but requires storage and bookkeeping |

# [4] Lost Reply Messages

- If no reply arrives, the client may retransmit the request.
- That creates the duplicate-request problem.
- Typical defenses:
  - attach a sequence number or request ID
  - remember recently executed requests
  - return the cached response for duplicates

## [5] Client Crashes

- A client may crash after sending a request but before receiving the reply.
- One approach is to force the client to log the request on stable storage before sending.
- Another approach is **reincarnation**: divide time into epochs and revoke old requests that outlive their epoch.

# From Communication Correctness to Atomic Commit

- RPC semantics answer: **did one request maybe execute?**
- Distributed commit answers a different question: **did all participants make the same final decision?**
- The property we now care about is **atomicity** across nodes.

# Distributed Commit — The Atomicity Problem

- Scenario: one transfer touches two different databases.

```
Client: TRANSFER $100 from AccountA (DB1) to AccountB (DB2)
  DB1: DEBIT  $100 from AccountA  ✓
  DB2: CREDIT $100 to  AccountB   ✗  (crash before commit)
```

- Without coordination, one side commits while the other does not.
- Requirement: **all participants commit, or all abort**.
- Distributed commit is the protocol family that enforces this guarantee.

# Distributed Commit: One-Phase Protocol

- A naive design lets one coordinator send `COMMIT` to everyone.
- This **one-phase commit** protocol is not fault tolerant enough.
- If a participant crashes before receiving `COMMIT`, the coordinator cannot distinguish:
  - the participant never saw the command
  - the participant saw it but failed before replying

> We need a voting step before the final decision.

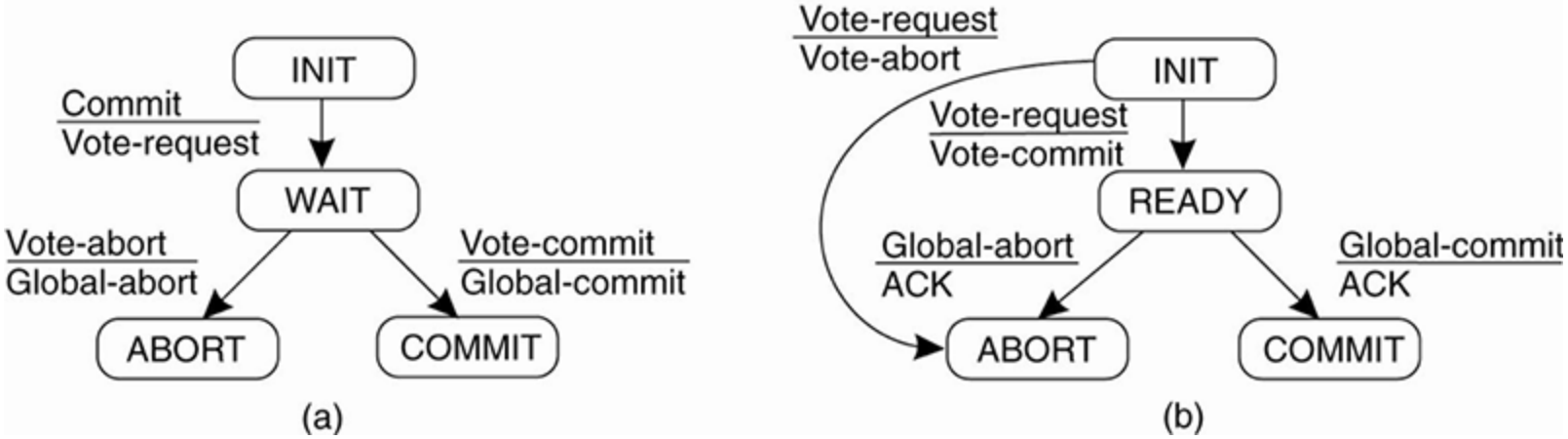
# Two-Phase Commit (1)

- Phase 1: **voting**
  - Coordinator sends `VOTE_REQUEST` to all participants.
  - Each participant responds with:
    - `VOTE_COMMIT` — “I am prepared to commit my part.”
    - `VOTE_ABORT` — “I cannot commit.”

# Two-Phase Commit (2)

- Phase 2: **decision**
  - If **all** participants voted `COMMIT`, coordinator sends `GLOBAL_COMMIT`.
  - If **any** participant voted `ABORT`, coordinator sends `GLOBAL_ABORT`.
- Participants wait for this final message before taking their final action.

# Two-Phase Commit (State Machines)




- (a) Coordinator state machine.
- (b) Participant state machine.

# Two-Phase Commit — Failure and Recovery Rules

| Local state after recovery | Safe action |
|---|---|
| `INIT` | Abort the transaction |
| `COMMIT` / `ABORT` | Re-enter that state and notify peers if needed |
| `READY` | Cannot decide alone; contact coordinator or peers; may still block |

- Coordinator recovery is log-based as well: after restart, it retransmits pending requests or the final decision.

# Two-Phase Commit — Blocking Problem

- Suppose every participant voted `COMMIT`.
- The coordinator crashes **after** collecting votes but **before** sending `GLOBAL_COMMIT` or `GLOBAL_ABORT`.
- Every participant is stuck in `READY`.
- No participant can decide safely on its own.

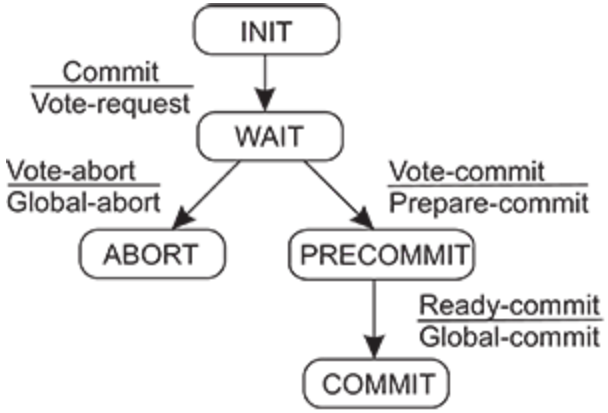
> 2PC preserves **atomicity**, but it may sacrifice **liveness**.

# Three-Phase Commit

- 3PC adds a `PRECOMMIT` phase between “ready” and “commit”.
- This removes the most dangerous ambiguous state from 2PC.
- But 3PC avoids blocking **only under stronger timing assumptions** than 2PC.
- In realistic asynchronous networks, those assumptions are much harder to justify.

# Three-Phase Commit (State Machines)

- Like 2PC, 3PC still uses a coordinator and participants.



# Three-Phase Commit — Why It Is Rare

- It sends more messages and maintains more states than 2PC.
- Its liveness argument depends on stronger synchrony assumptions.
- So 3PC is useful mainly as a conceptual lesson:
  - **liveness depends on the failure model**
  - adding a phase does not magically fix asynchrony

> We now switch questions: from **"can one transaction commit atomically?"** to **"can replicas maintain one shared history?"**

# Atomic Commit vs. Consensus

| Question | 2PC / 3PC | Paxos / Raft |
|---|---|---|
| What is decided? | `COMMIT` or `ABORT` for one transaction | next log entry / command |
| Who participates? | heterogeneous participants with private local state | replicas of one service |
| Progress condition | everyone follows the commit protocol; one `VOTE_ABORT` aborts | a majority quorum can keep making progress |
| Failure pain point | coordinator failure may block; 3PC needs stronger timing assumptions | leader can be replaced; quorum intersection preserves safety |
| Typical use | cross-shard / cross-service atomicity | replicated state machine, control plane, leader election |

# Why 2PC / 3PC Do Not Replace Paxos / Raft

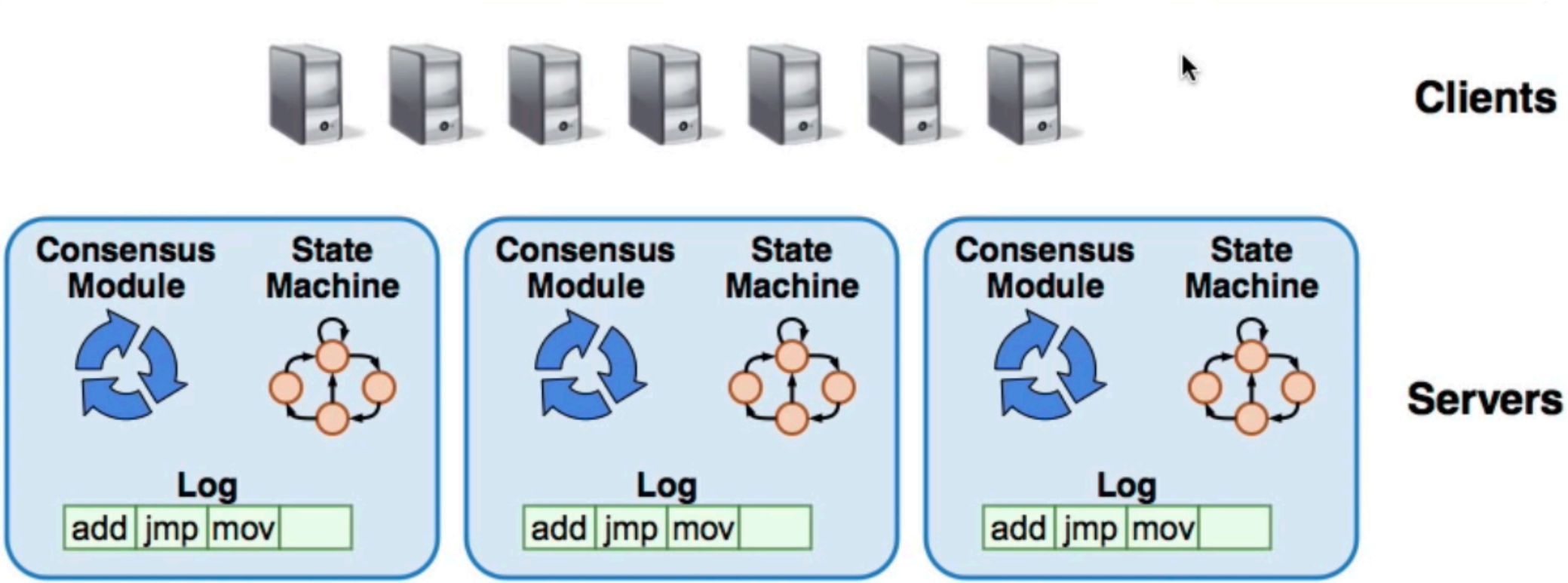
- 2PC / 3PC solve **atomic commit**, not replicated-log agreement.
- They do not give a control plane an automatic way to keep serving after a coordinator or leader-like node fails.
- 2PC needs all relevant participants in the decision path; Paxos / Raft need only a functioning **majority**.
- 3PC changes the coordinator protocol, but it still does not become a replicated log or a self-healing leader-election mechanism.

> Atomicity across participants is not the same property as fault-tolerant agreement among replicas.

# Why Paxos / Raft Do Not Replace 2PC

- In atomic commit, each participant has a real **veto** based on local state:
  - constraint violation
  - lock conflict
  - disk or log failure
- Paxos / Raft replicas do not vote based on private business feasibility; they choose one value for a replica group.
- A majority deciding `COMMIT` is useless if one participant physically cannot commit.
- The common architecture is therefore layered:
  - **within** each replica group: Paxos / Raft for availability
  - **across** groups: 2PC for atomic commit

# Paxos: Goal — Replicated Log




- Replicated log -> replicated state machine.
- Assumptions: **crash-stop failures**; messages may be delayed, reordered, or duplicated.
- **Safety** does not depend on timing.
- **Liveness** needs a functioning majority and eventual synchrony / stable leadership.

# Basic Paxos vs. Multi-Paxos

- **Basic Paxos**
  - One or more proposers try to choose a **single value**.
  - Only one value may ultimately be chosen.
- **Multi-Paxos**
  - Run many instances of Basic Paxos to fill a replicated log.
  - Reuse a stable leader to reduce conflict and overhead.

# Paxos Requirements

- **Safety**
  - Only a single value may be chosen.
  - A server must never learn a false decision.
- **Liveness**
  - Eventually some proposal is chosen.
  - Eventually servers learn that choice.

> Keep them separate: Paxos safety survives much worse timing conditions than Paxos liveness.

## Paxos Components

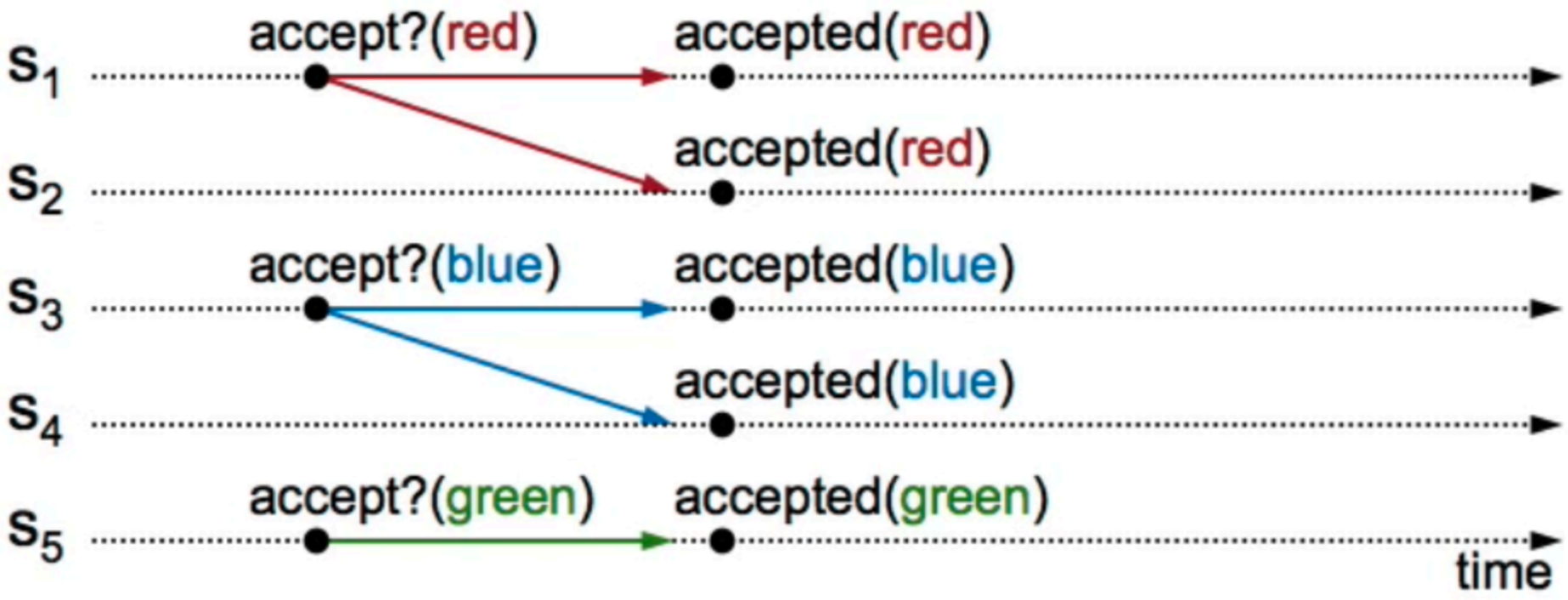
- **Proposers** put forward candidate values.
- **Acceptors** vote on proposals and remember accepted state.
- In many implementations, each physical server contains both roles.
- Learners are often implicit: replicas eventually learn the chosen value from the protocol state.

## Strawman: Single Acceptor

- Simplest idea: let one acceptor decide.
- If that acceptor crashes, the decision is lost.
- So Paxos uses a **quorum**:
  - multiple acceptors
  - a value is chosen when accepted by a **majority**

# Quorum Problem: Split Votes

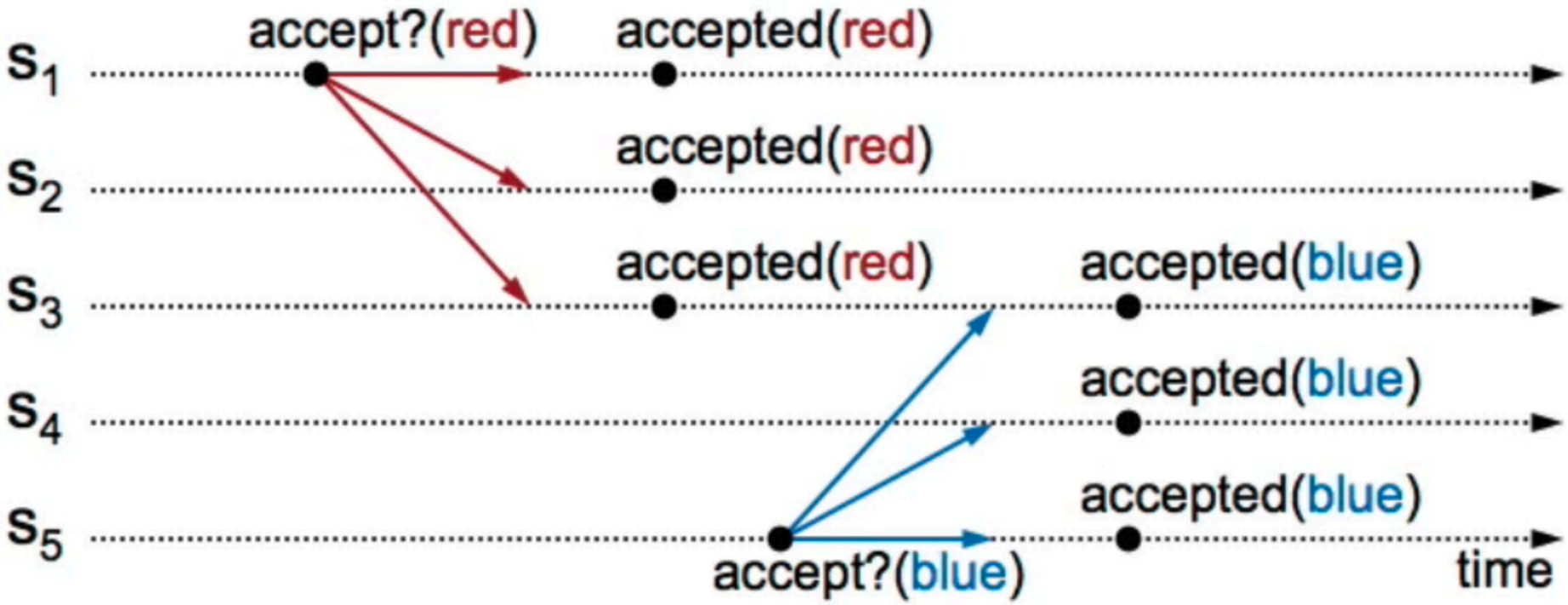
- Suppose each acceptor accepts only the **first proposal** it sees.
- Then simultaneous proposals can lead to deadlock: no proposal gets a majority.




> Acceptors must sometimes accept more than one proposal over time.

# Quorum Problem: Conflicting Choices

- Now suppose acceptors accept **every proposal** they receive.
- Then two different values might each gather a majority at different times.




> Once a value has been chosen, later proposals must preserve that choice.

# Ordering Proposals with Proposal Numbers

- Each proposal carries a unique number `n`.
- Higher-numbered proposals take priority over lower-numbered ones.
- A proposer must generate numbers that never go backward, even after restart.
- One simple scheme:
  - `(round, server_id)`
  - persist the largest round used or observed

# Basic Paxos

Two phases:

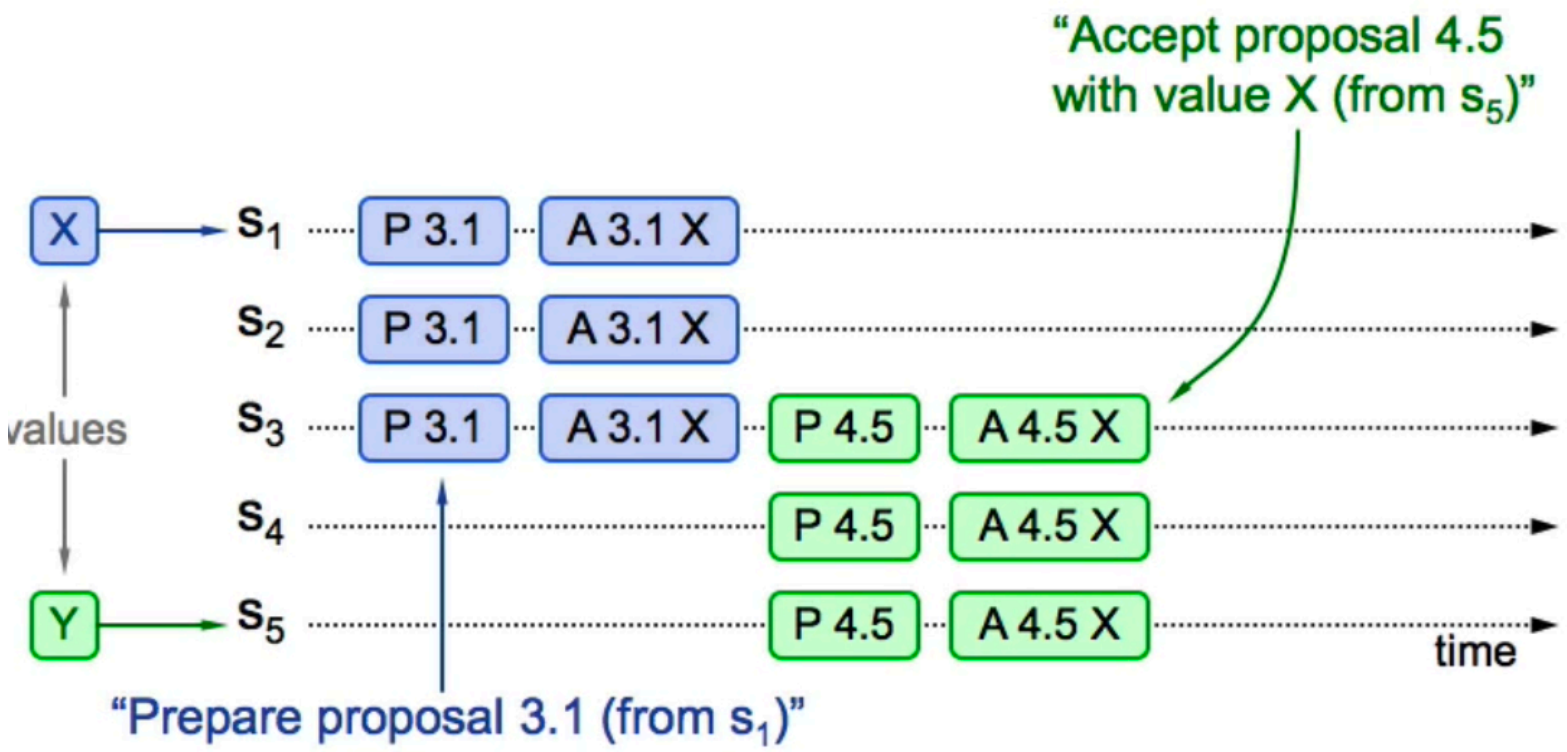
- **Prepare**
  - block older proposals
  - learn about any value that may already be chosen
- **Accept**
  - ask acceptors to accept a specific value

# Basic Paxos (Steps)

1. Proposer chooses a new proposal number `n`.
2. It broadcasts `Prepare(n)`.
3. Each acceptor promises not to accept lower-numbered proposals and returns its highest accepted proposal/value, if any.
4. If the proposer receives responses from a **majority**, it must adopt the value from the highest-numbered prior acceptance, if one exists.
5. It then broadcasts `Accept(n, value)`.
6. Acceptors accept if `n` is still high enough.
7. When a **majority** accepts, the value is chosen.

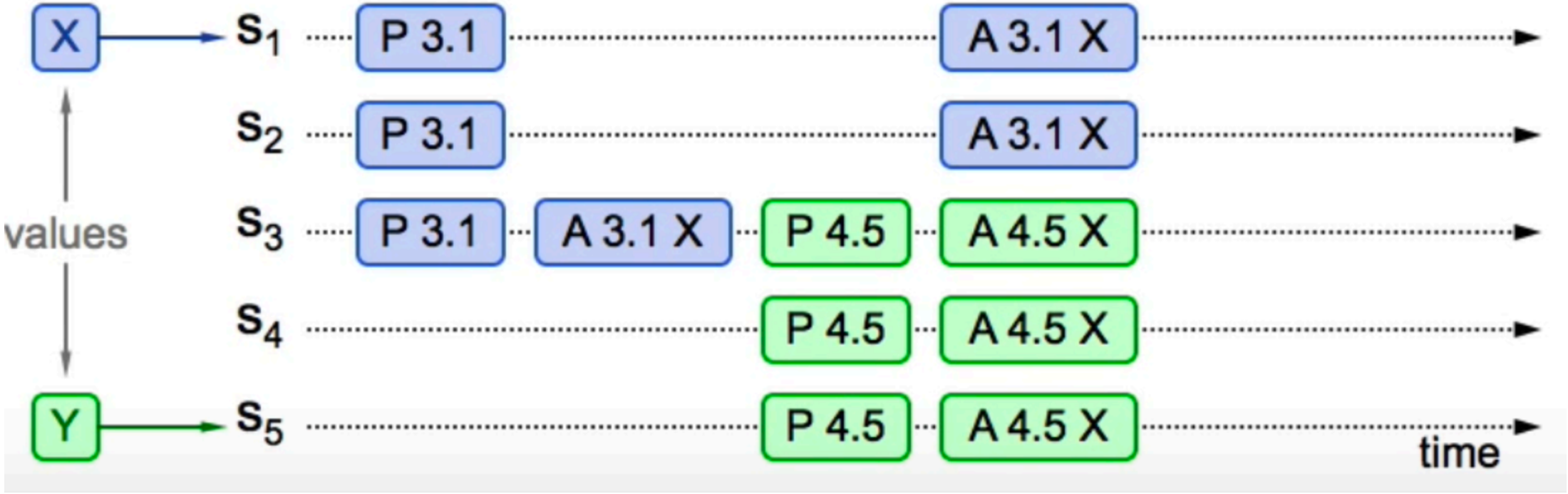
# Basic Paxos Example: Previously Chosen Value

- A later proposer discovers that some value was already effectively chosen and must keep it.



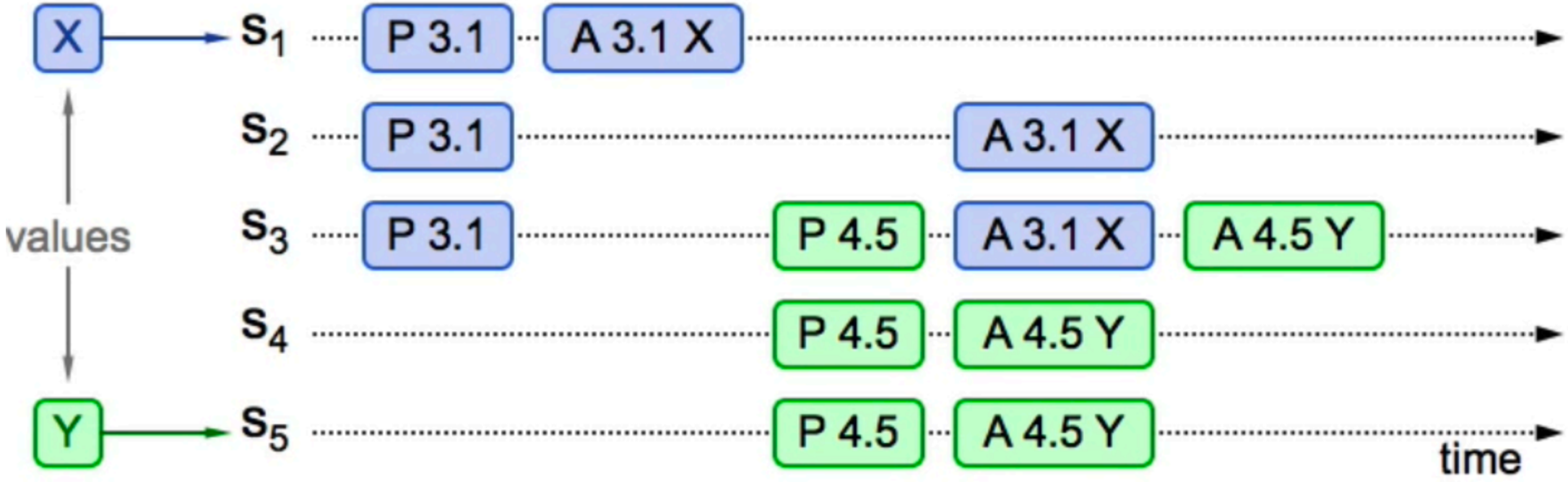
# Basic Paxos Example: Previously Accepted but Not Yet Chosen

- A later proposer may discover an earlier accepted value that never became chosen.
- If that is the highest-numbered accepted value returned by its prepare quorum, it must carry that value forward.

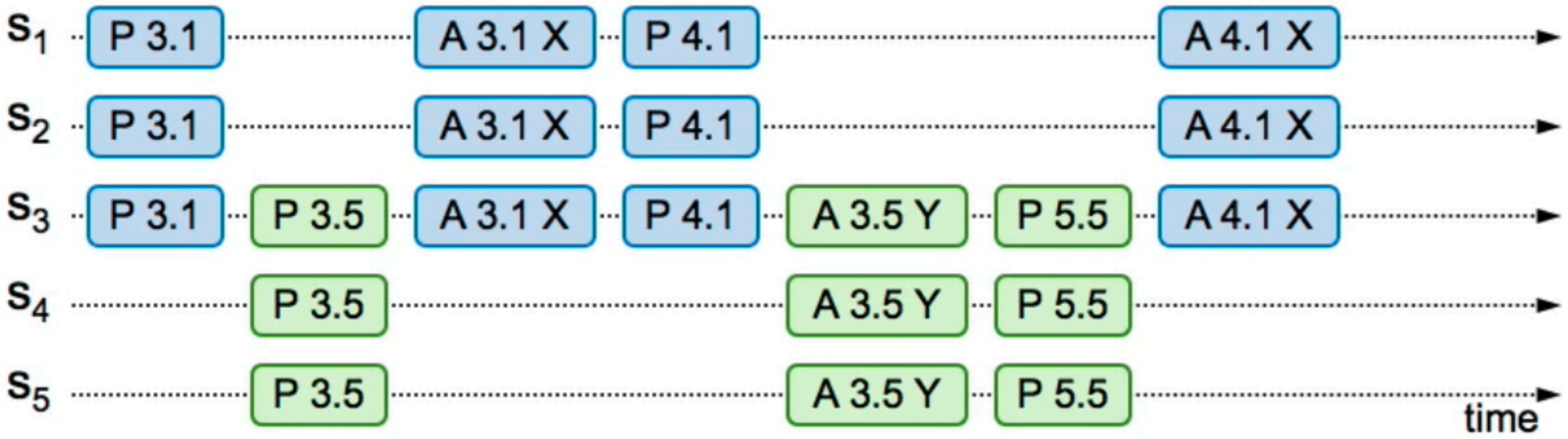


# Basic Paxos Example: Older Proposal Gets Pre-empted

- If a newer proposal number appears, an older proposer may have to retry.
- This preserves safety, but it may hurt progress.

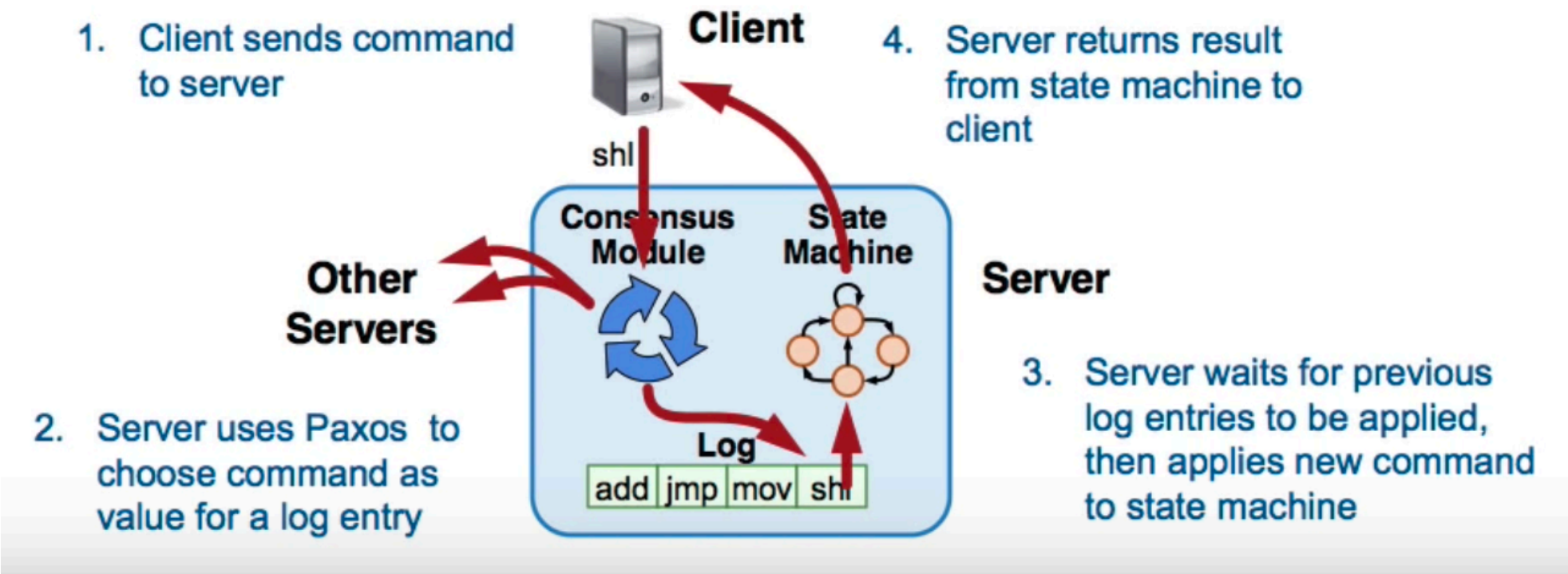


# Paxos Liveness




- Competing proposers can livelock by repeatedly pre-empting each other.
- Randomized backoff helps.
- Multi-Paxos typically solves this operationally with a stable leader.

# Multi-Paxos: Basic Idea




- Basic Paxos chooses **one value for one slot**.
- Multi-Paxos repeats that idea to build a **replicated log**.
- A **stable leader** usually proposes commands for successive slots.
- Majority quorums still provide safety; leadership mainly improves performance and liveness.

> Short version: Basic Paxos is one decision; Multi-Paxos is the same safety idea turned into a practical log.

# Multi-Paxos: Why It Is Practical

- When leadership stabilizes, the leader usually avoids a full fresh prepare phase for every slot.
- In the common case, the leader proposes the next log entry and acceptors vote on it.
- If the leader crashes, another node can start a higher-numbered round and take over.
- The log may contain old accepted entries, so a new leader must first recover and respect any prior safe history.

> Teaching takeaway: **stable leader for speed, majority quorum for safety, replicated log for state-machine replication.**

# 2PC Coordinator Is Also Consensus-Backed

```
                one cross-shard transaction T

   Replica Group A                          Replica Group B
 ┌──────────────────────┐<---- 2PC ----->┌──────────────────────┐
 │ leader               │                 │ leader               │
 │ = coordinator for T  │                 │ = participant for T  │
 │ Paxos / Raft inside  │                 │ Paxos / Raft inside  │
 │ followers replicate  │                 │ followers replicate  │
 │ coordinator state    │                 │ participant state    │
 └──────────────────────┘                 └──────────────────────┘
```

- There is **no extra cross-group leader election** for 2PC itself.
- For transaction `T`, an already-existing group leader often acts as the **2PC coordinator**.
- In this example, Group A's followers replicate the coordinator state via **Paxos / Raft**; Group B replicates its participant state the same way.
- Without replicated coordinator state, classic 2PC can block after coordinator failure.
- Some systems use a dedicated transaction-manager group instead; the same replication idea still applies.

> This is still the **crash-fault** world: 2PC handles atomic commit across groups, while Paxos / Raft keep each group highly available.

## Contrast: Byzantine Faults

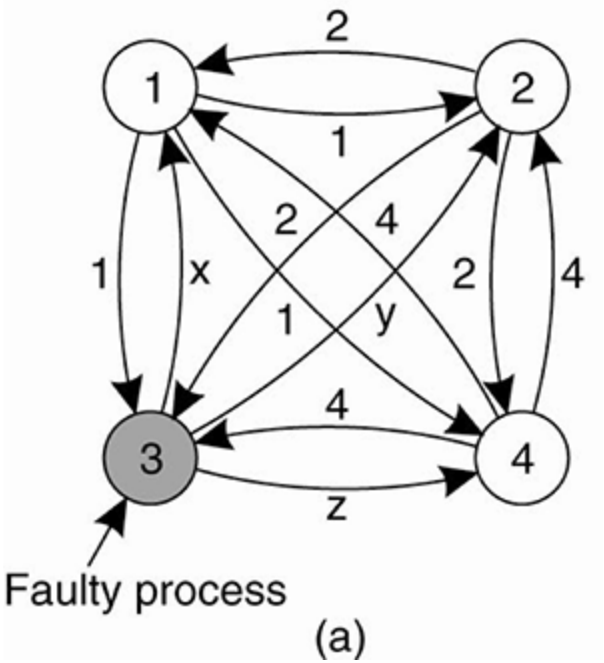
- So far we assumed **crash-stop / omission** behavior.
- If nodes can send inconsistent, arbitrary, or malicious messages, the protocol family changes.
- Byzantine agreement is **not** the next step after 2PC or Paxos; it is a different branch for a stronger failure model.

> For Kubernetes-style control planes, the main path is still crash-fault consensus, not Byzantine consensus.

# Byzantine Agreement: Problem Statement

- Assume `N` processes, and up to `f` may be Byzantine.
- In this teaching sketch, each process `i` contributes a value `v_i`.
- Goal: all correct processes construct the same system-wide interpretation, even if faulty nodes send conflicting information.
- For a correct process `i`, the agreed interpretation should preserve `v_i`; faulty processes may be marked unknown.

# Byzantine Agreement: Step 1 — Broadcast Initial Values




- Each process sends its claimed value to every other process.
- If all processes are honest, everyone should receive a consistent set of claims.
- A Byzantine node may already diverge here by sending different values to different peers.

# Byzantine Agreement: Step 2 — Local Views Diverge

1 Got(1, 2, x, 4)
2 Got(1, 2, y, 4)
3 Got(1, 2, 3, 4)
4 Got(1, 2, z, 4)

(b)

- After Step 1, each process assembles a local vector of what it heard.
- These vectors may already disagree across correct processes.
- The disagreement is caused by inconsistent reports from Byzantine participants.

# Byzantine Agreement: Step 3 — Exchange Observations

| 1 Got | 2 Got | 4 Got |
|---|---|---|
| (1, 2, y, 4) | (1, 2, x, 4) | (1, 2, x, 4) |
| (a, b, c, d) | (e, f, g, h) | (1, 2, y, 4) |
| (1, 2, z, 4) | (1, 2, z, 4) | (i, j, k, l) |

(c)

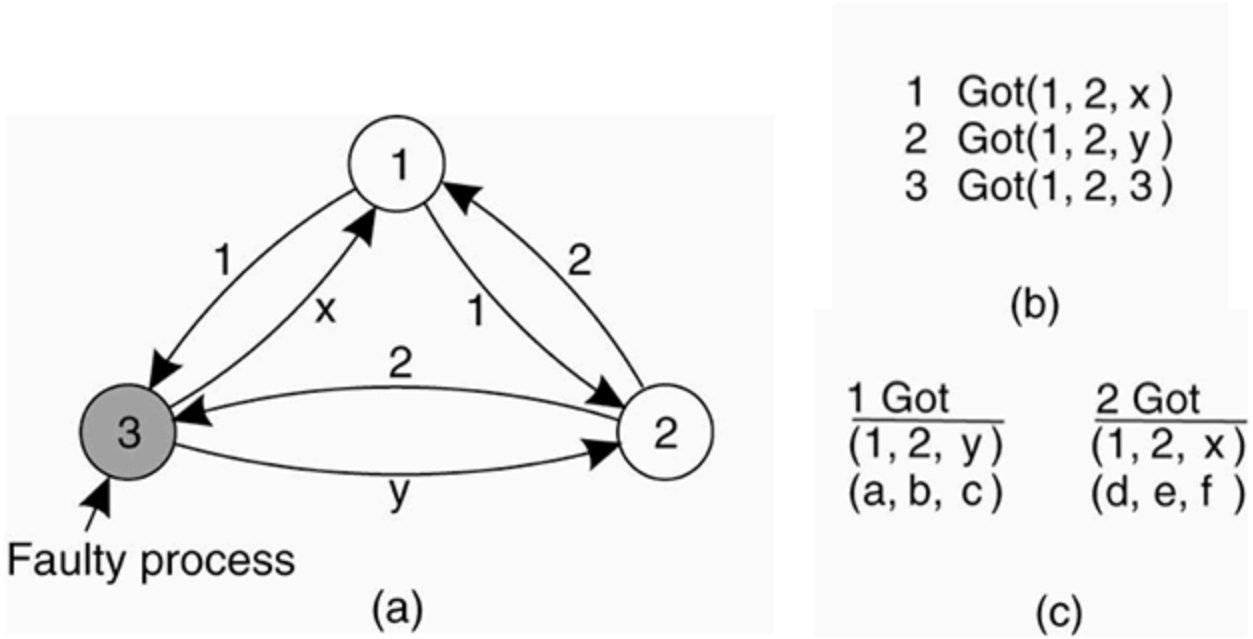
- Each process now forwards its assembled vector to every other process.
- Correct processes compare not only direct claims, but also what others observed.
- This extra round creates the redundancy needed to reason about who is inconsistent.

> This is a pedagogical sketch, not the full oral-messages protocol.

# Byzantine Agreement: Step 4 — Element-wise Majority Reasoning

- Each process examines the `i` th element across the newly received vectors.
- If some value has a majority, record that value in the result vector.
- If there is no clear majority, mark that position as **UNKNOWN**.
- In the teaching example, correct processes can still agree on the entries contributed by correct peers.

# Byzantine Agreement: Lower Bound




- Classic result: to tolerate `f` Byzantine faults, we need at least `n >= 3f + 1` processes.
- Equivalently, agreement requires at least `2f + 1` **correct** processes.
- Majority voting alone is not enough when a faulty node can send different lies to different peers.
- This higher replication cost is one reason BFT systems are heavier than crash-fault consensus systems.

# Where These Protocols Show Up

| System need | Typical mechanism |
|---|---|
| Reliable request/response between components | TCP, RPC retry, deduplication |
| All-or-none update across participants | 2PC-style commit |
| Replicated control-plane state | Paxos / Raft-style consensus |
| Agreement under arbitrary or malicious behavior | BFT protocols |

- etcd and similar control planes live mainly in the **crash-fault consensus** world.

# Summary

| Failure model | Required guarantee | Protocol family | Role in this course |
|---|---|---|---|
| Crash / omission | Correct request/response | TCP + RPC retry / dedup | Main path |
| Crash / omission | Atomic all-or-none update | 2PC, 3PC | Main path |
| Crash-stop + async messaging | Replicated agreement | Paxos / Multi-Paxos | Main path |
| Byzantine faults | Agreement under arbitrary behavior | Byzantine Agreement / BFT | Contrast branch |

> Fault tolerance is a **map** from failure models and required guarantees to protocol families.